

Architektury systemów komputerowych: Reference

Data: 2026-05-05

Zakres: Do wykładu 9 włącznie

1. Rejestry x86_64

<code>%rax</code>	<code>%eax</code>	<code>%ax</code> <code>%ah %al</code>	<code>%rbx</code>	<code>%ebx</code>	<code>%bx</code> <code>%bh %bl</code>
<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code> <code>%ch %cl</code>	<code>%rdx</code>	<code>%edx</code>	<code>%dx</code> <code>%dh %dl</code>
<code>%rsi</code>	<code>%esi</code>	<code>%si</code> <code>%sil</code>	<code>%rdi</code>	<code>%edi</code>	<code>%di</code> <code>%dil</code>
<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code> <code>%bpl</code>	<code>%rsp</code>	<code>%esp</code>	<code>%sp</code> <code>%spl</code>
<code>%r8</code>	<code>%r8d</code>	<code>%r8w</code> <code>%r8b</code>	<code>%r9</code>	<code>%r9d</code>	<code>%r9w</code> <code>%r9b</code>

Uwaga: Rejestry od %r10 do %r15 używają schematu: `%r10`, `%r10d`, `%r10w`, `%r10b`.

Rejestry ogólnego przeznaczenia

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną z funkcji.
<code>%rbx</code>	Rejestr bazowy. Callee-saved.
<code>%rcx</code>	Licznik w pętlach i przesunięciach bitowych. 4. argument .
<code>%rdx</code>	Rejestr danych. Używany w mnożeniu/dzieleniu. 3. argument .

Indeksy i wskaźniki

<code>%rdi</code>	Indeks docelowy (DI). 1. argument .
<code>%rsi</code>	Indeks źródłowy (SI). 2. argument .
<code>%rsp</code>	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki. Modfikowany przez <code>push</code> / <code>pop</code> / <code>call</code> / <code>ret</code> .
<code>%rbp</code>	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.

Nowe i specjalne rejestry

<code>%r8-9</code>	Kolejno: 5. i 6. argument . Caller-saved.
<code>%r10</code> oraz <code>%r11</code>	Rejestry tymczasowe. Caller-saved.
<code>%r12</code> do <code>%r15</code>	Ogólnego przeznaczenia. Wszystkie callee-saved .
<code>%rip</code>	Wskaźnik instrukcji (IP). Następna instrukcja. Modyfikowany przez skoki i wywołania.
<code>%rflags</code>	Rejestr flag statusu. Zmieniany przez <code>cmp</code> , <code>test</code> .

Rejestry wektorowe (zmiennoprzecinkowe)

<code>%xmm0</code> do <code>%xmm15</code>	128-bitowe rejestry. <code>%xmm0</code> to wartość zwracana (float/double). <code>%xmm0-%xmm7</code> to pierwsze 8 argumentów . Wszystkie są caller-saved .
<code>%ymm0</code> do <code>%ymm15</code>	256-bitowe rozszerzenie rejestrów XMM. Dzielą z nimi najmłodsze 128 bitów.

2. Flagi stanu (Rejestr %rflags)

<code>ZF</code>	Zero. Wynik to 0 (np. argumenty są równe).
<code>SF</code>	Sign. Wynik jest ujemny (MSB = 1).
<code>CF</code>	Carry. Przepełnienie dla liczb bez znaku (unsigned).
<code>OF</code>	Overflow. Przepełnienie dla liczb ze znakiem (signed).

3. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Immediate)	<code>\$Imm</code>	<code>Imm</code>
Rejestrowy (Register)	<code>%Ra</code>	<code>%Ra</code>
Bezpośredni (Direct)	<code>Imm</code>	<code>Mem[Imm]</code>
Pośredni (Indirect)	<code>(%Rb)</code>	<code>Mem[%Rb]</code>
Z przesunięciem (Indirect displacement)	<code>D(%Rb)</code>	<code>Mem[%Rb + D]</code>
Skalowany (Indirect scaled-index)	<code>D(%Rb, %Ri, S)</code>	<code>Mem[%Rb+%Ri*S+D]</code>
Skalowany (baseless)	<code>(, %Ri, S)</code>	<code>Mem[%Ri * S]</code>

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko wartości: 1, 2, 4 lub 8).

4. Konwencja wywoływań (System V AMD64 ABI)

Przekazywanie argumentów

`%rdi` → `%rsi` → `%rdx` → `%rcx` → `%r8` → `%r9`

Argumenty 7+ Na stosie od końca.
Wynik Zawsze w `%rax`.

Ochrona rejestrów (ABI)

Caller-saved
(Można niszczyć)

`%rax`, `%rdi`, `%rsi`,
`%rdx`, `%rcx`, `%r8` - `%r11`

Callee-saved
(Musi przywrócić)

`%rbx`, `%rbp`, `%r12` - `%r15`

Ramka Stosu

Start (alokacja)

```
push %rbp
mov %rsp, %rbp
sub $32, %rsp
```

Koniec (sprzątanie)

```
leave
ret
/* równowaznie:
* mov %rbp, %rsp
* pop %rbp
* ret */
```

5. Struktury i wyrównanie

1. **Wyrównanie pola (K):** Zmienna K -bajtowa pod adresem $p \bmod K = 0$.
2. **Rozmiar całkowity:** Całkowity `sizeof` struktury musi być podzielny przez największe wyrównanie w strukturze ($K_{\max}$).

```
struct {
    char c; // 1 bajt (offset 0)
           // [3 bajty wewn. paddingu]
    int i; // 4 bajty (offset 4)
    short s; // 2 bajty (offset 8)
           // [2 bajty zewn. paddingu]
} // K_max = 4. Rozmiar: 1+3+4+2+2 = 12 bajtów.
```

6. Najważniejsze instrukcje (AT&T)

Opcode	Efekt	Opis
Przesyłanie danych		
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos.
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ <code>pop %rbp</code>	Sprząta ramkę stosu.
Adresowanie i arytmetyka		
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>inc / dec D</code>	$D \leftarrow D \pm 1$	Inkrementacja / dekrementacja.
<code>neg D</code>	$D \leftarrow -D$	Negacja arytmetyczna (jak w U2).
Dzielenie (zawsze w parze)		
<code>cqto</code>	$\%rdx:\%rax \leftarrow \text{sgn_ext}(\%rax)$	Uwaga: Rozszerza $\%rax$ do 128-bit przed <code>idivq</code> .
<code>idiv S</code>	$\%rax \leftarrow \text{wynik}$ $\%rdx \leftarrow \text{reszta}$	Dzielenie <code>%rdx:%rax</code> przez S. Wynik w <code>%rax</code> , reszta w <code>%rdx</code> .
Logiczne i bitowe		
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często używane jako <code>xor %rax, %rax</code> do zerowania.
<code>not D</code>	$D \leftarrow \sim D$	Negacja bitowa (odwrócenie bitów).

Opcode	Efekt	Opis
<code>sar / shl k, D</code>	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
<code>shr k, D</code>	$D \gg = k$	Logiczne w prawo. Dopełnia zerami.
Porównania i skoki		
<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND (np. test czy rejestr jest zerem).
<code>jX dest</code>	$\text{if}(X) \ \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia bajt (np. <code>sal</code>) na 0 lub 1 na podstawie flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (optymalizacja zamiast skoku).
<code>call / ret</code>		Skok do funkcji / Powrót (obsługa stosu i <code>%rip</code>).
Operacje wektorowe		
<code>movaps / movups S, D</code>	$D \leftarrow S$	Kopiuje 128-bitów. aps (aligned) - wymaga wyrównania w pamięci, ups nie.
<code>movsd / movss S, D</code>	$D \leftarrow S$	Kopiuje pojedynczą wartość skalarną (<code>double</code> / (s) <code>float</code>).
<code>add/sub/mul/div pd/ps</code>	$D \leftarrow D \text{ op } S$	Arytmetyka wektorowa. Wykonuje operację na wielu parach równocześnie.
<code>add/sub/mul/div sd/ss</code>	$D \leftarrow D \text{ op } S$	Arytmetyka skalarna. Modyfikuje tylko najmłodszą wartość w rejestrze.
<code>vxorps S1, S2, D</code>	$D \leftarrow S2 \wedge S1$	Wektorowy XOR .
<code>v... (np. vmulss)</code>	$D \leftarrow S2 \text{ op } S1$	Przedrostek v wprowadza 3 argumenty (źródło 1, źródło 2, cel). Nie niszczy danych wejściowych.
<code>vfmadd231ss S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add. Mnoży i dodaje. Cyfry 231 określają, że mnożymy dwa źródła, a wynik dodajemy do D .

Uwaga o sufiksach: Instrukcje mogą przyjmować sufiks określający rozmiar danych:
b (byte, 8-bit), **w** (word, 16-bit), **l** (long, 32-bit), **q** (quadword, 64-bit).
Np. `movq` kopiuje 64 bity, a `movl` kopiuje 32 bity (i zeruje górną połowę rejestru!).

7. Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

Sufiks	Synonim	Znaczenie / Warunek
Równość i znak (<code>ZF</code> , <code>SF</code>)		
<code>e</code>	<code>z</code>	Equal / Zero (równe / wynik to 0)
<code>ne</code>	<code>nz</code>	Not Equal / Not Zero (nierówne)
<code>s</code>		Sign (wynik ujemny, <code>SF=1</code>)
<code>ns</code>		Not Sign (wynik dodatni lub 0, <code>SF=0</code>)
Liczby ze znakiem (signed)		
<code>g</code>		Greater
<code>ge</code>		Greater or Equal
<code>l</code>		Less
<code>le</code>		Less or Equal
Liczby bez znaku (unsigned)		
<code>a</code>		Above (powyżej: <code>></code>)
<code>ae</code>	<code>nc</code>	Above or Equal / No Carry (większe równe: <code>>=</code>)
<code>b</code>	<code>c</code>	Below / Carry (poniżej: <code><</code>)
<code>be</code>		Below or Equal (mniejsze równe: <code><=</code>)

8. Optymalizacje i atrybuty

Podstawowe techniki	
Strength red.	Zamiana drogich operacji na tańsze (np. <code>mul</code> → <code>lea</code> / <code>shl</code>).
Loop unroll.	Rozwinięcie pętli (np. skok co 4. iterację). Mniej skoków = mniejszy narzut.
Code motion	Wyrzucenie obliczeń niezmienników (np. stałych w pętli) przed pętlę.
Common subexpr. elim.	Obliczenie wspólnego fragmentu tylko raz.
Const folding	Obliczanie stałych wyrażeń (np. <code>2+2</code>) w czasie kompilacji.
Branch predict	Sprzętowe przewidywanie skoków. By unikać kar, można użyć <code>cmovX</code> .
Inlining	Wklejenie ciała funkcji w miejsce wywołania. Eliminuje narzut <code>call</code> / <code>ret</code> .
Analiza pamięci i atrybuty GCC	
Aliasing	Problem nakładania się wskaźników. Keyword <code>restrict</code> , pomaga w optymalizacji.
pure / const	Funkcja bez skutków ubocznych (<code>const</code> dodatkowo nie czyta zmiennych globalnych).

9. Bezpieczeństwo kodu i exploity

Zagrożenia i błędy	
Buffer overflow	Zapis poza limit bufora. Nadpisuje sąsiadującą pamięć.
Seg fault	Próba dostępu bez uprawnień (np. odczyt <code>NULL</code> , modyfikacja read-only).
Stack smashing	Atak nadpisujący adres powrotu na stosie, by przejąć kontrolę po <code>ret</code> .
ROP (Gadżety)	Łączenie legalnych instrukcji kończących się <code>ret</code> w celu ominięcia zabezpieczeń.
Mechanizmy obronne	
Stack canaries	Losowa wartość ułożona przed adresem powrotu. Weryfikowana przed <code>ret</code> .
Nonexec code segments	Stos dostaje sprzętowy zakaz wykonywania kodu (tylko R/W).
Rand. stack offsets	Losowanie adresów stosu przy każdym starcie.